Object-Oriented Programming

2. Subtyping polymorphism

Filip Křikava

https://courses.fit.cvut.cz/BI-OOP/

Any questions?

A object has state, behavior, and identity

```
class Counter:
  private var count: Int = 0
  def get: Int = count
  def inc(): Unit = count += 1

val a = new Counter()
val b = Counter()

a.inc()

a.get           // 1
b.get           // 0
a eq b          // false
```

- State:
  - each instance stores its own count.
- Behavior:
  - `get` reads
  - `inc()` changes it
- Identity:
  - `a` and `b` refer to different objects.
- The class defines their common structure and behavior.

Constructor and members

```
class Counter(init: Int = 0,
              val step: Int = 1):

  require(init >= 0 && step > 0)

  private var count: Int = init

  def get: Int = this.count

  def set(count: Int): Unit =
    require(count >= 0)
    this.count = count

  def inc(): Unit =
    this.count += this.step
```

- `Counter` names both a class and a type.
- The header defines the primary constructor.
- `init` supplies the initial state; defaults allow omission.
- `val step` exposes a read-only member.
- `private var count` stores mutable state.
- `def` introduces each method.

A parameter is not necessarily a public field

```
class Counter(init: Int = 0,
              val step: Int = 1):
  private var count: Int = init
  def get: Int = count
  def inc(): Unit = count += step
  // ...

val c = new Counter(init = 2, step = 3)
c.step                    // 3
c.get                     // 2
c.inc()
c.get                     // 5

// c.init                 // not public
// c.count                // private
// c.step = 4             // read-only
```

- `init`: constructor input, available inside the class.
- `val step`: readable, but cannot be reassigned.
- `var count`: mutable
    - `private` restricts access.
- A constructor parameter declared `var` also exposes a setter.
- Encapsulation lets methods validate changes.

A method and its receiver: this

```
class Counter(init: Int = 0,
              val step: Int = 1):

  private var count: Int = init

  def set(count: Int): Unit =
    require(count >= 0)
    this.count = count

  def get: Int = this.count

  def inc(): Unit =
    this.count += this.step

  // ...

val c = new Counter()
c.set(5)
c.inc()
c.get                          // 6
```

- For `c.set(5)`, `this` is the object referenced by `c`.
- `this.count` is its field; the unqualified `count` is the parameter.
- `Int` is the parameter/result type; `Unit` is the effect-only result.
- The last expression is the result; no `return` is needed.
- Convention: `get` reads a property, `inc()` has effects.

Instantiation: a fresh object and a reference

```
class Counter(init: Int = 0,
              val step: Int = 1):
  private var count: Int = init
  def get: Int = count
  // ... validation, set, inc

val c: Counter = new Counter(2, 3)
c.get                          // 2
c.step                         // 3

val d = Counter(init = 2, step = 3)

val e = new Counter()

e.get                          // 0
e.step                         // 1
c eq d                         // false
```

- A `new` expression allocates and initializes an instance.
- The result is a reference, bound here to `c`.
- Arguments supply constructor parameters.
- The class body performs initialization for each instance.
- For this class, Scala 3 allows omitting `new`.
- I don't need to specify the variable type, it will be inferred

Construction runs the class body

```
class Counter(init: Int = 0,
              val step: Int = 1):

  println("constructing")
  private var count: Int = init
  println("ready: " + count)

  def inc(): Unit =
    println("incrementing")
    count += step
  // ... validation, get, set

val c = new Counter(2)
// constructing
// ready: 2

c.inc()
// incrementing
```

- Field initializers and statements run in source order.
- The body of `inc()` waits until the method is called.
- Another `new Counter(...)` repeats construction.
- Validation can reject construction by throwing an exception.
- An auxiliary `def this(...)` must delegate to another constructor eventually terminating the primary constructor

val fixes the reference, not the object

```
class Counter(init: Int = 0,
              val step: Int = 1):
  private var count: Int = init
  def get: Int = count
  def inc(): Unit = count += step
  // ... validation and set

val a = new Counter(1)
a.inc()
a.get                          // 2
// a = new Counter(9)          // not allowed

var b = a
b = new Counter(9)
b.inc()
a.get                          // 2
b.get                          // 10
```

- Mutating an object and reassigning a variable are different.
- `val a` fixes the reference stored in `a`.
- The Counter remains mutable because its count is a `var`.
- `var b` can hold another reference later.
- Assignment copies a reference, not the object.

Scala object: one named instance

```
class Counter(init: Int = 0,
              val step: Int = 1):
  // ... state and instance methods

object Counter:
  def zero(): Counter = new Counter()
  val SharedCounter = new Counter()

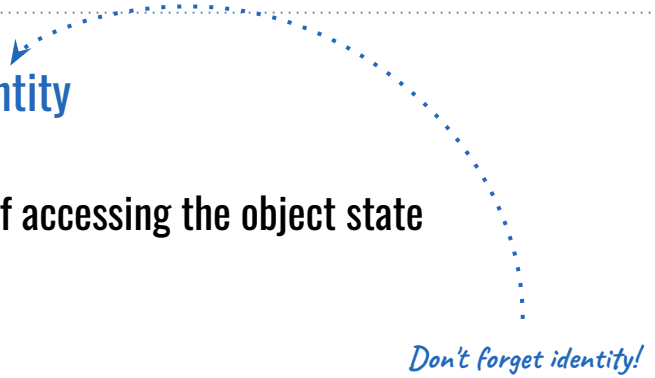
val a = Counter.zero()
val b = Counter.zero()
a eq b                          // false

val x = Counter.SharedCounter
val y = Counter.SharedCounter
x eq y                          // true
```

- `class Counter` describes instances made with `new`.
- `object Counter` defines one named singleton instance.
- With the same name, it is the class's companion object.
- Calling its factory can still create many Counters.
- `object` is placeholder for static items

Summary - what is an object?

- An object is a structure with a state, behavior and identity
  - state - instance variables (fields)
  - behavior - (instance) methods that alone are capable of accessing the object state
  - recursive - they can refer to themselves



- In the class-based system, it is a runtime instance of some class

- Objects communicate with one another by message passing
  - in the mainstream OOP sending a message is essentially a method invocation

- In this course we will assume reference semantics
  - all objects are implicitly references
  - this is the case of Java, Scala, Kotlin, JavaScript, Python, and other managed languages

Summary - what is a class?

- A blueprint (a template) for creating objects
  - possibly extensible
  - defines object's runtime structure
  - defines class members (fields, methods, constructors)

- Provide initial values for its state and implementation of its behavior
  - all objects created from the same class share the same methods, but contain separate copies of their instance variables (fields)

- In class-based languages it defines a type
  - in statically-typed languages it is a compile-time type

- In this course we will focus primarily on class-based languages

Inheritance and dispatch

Extend the counter - DoubleCounter

- `DoubleCounter` reuses (extends) `Counter's` state, initialization, get, and set.
- It changes inc
- the inherited private count remains inside the same object, invisible to `DoubleCounter`
- In construction we need to supply arguments to super class constructor

```
class Counter(init: Int = 0,
              val step: Int = 1):

  private var count: Int = init

  def get: Int = count

  def inc(): Unit = count += step
  // ...
```

```
class DoubleCounter(init: Int = 0, step: Int = 1)
  extends Counter(init, step):

  override def inc(): Unit =
    super.inc()
    super.inc()
```

A declared type is a view of the same object

```
class Counter:
  private var count: Int = 0
  def get: Int = count
  def inc(): Unit = count += 1
  // ...

class DoubleCounter extends Counter:
  override def inc(): Unit =
    super.inc()
    super.inc()

val d: DoubleCounter = new DoubleCounter(1)
val c: Counter = d

c eq d                      // true
c.inc()
c.get                       // 3
d.get                       // 3
```

- Static type of `c: Counter`.
- Runtime class of `c: DoubleCounter`.
- The static type controls which members are available.
- The runtime receiver selects an overridden body.
- Upcasting changes the view, not the object.

this and super

```
class Counter:
  private var count: Int = 0

  def get: Int = count
  def inc(): Unit = count += 1
  def twice(): Unit =
    this.inc()
    this.inc()
  // ...

class DoubleCounter extends Counter:
  override def inc(): Unit =
    super.inc()
    super.inc()

val c: Counter = new DoubleCounter()
c.inc()
c.twice()
```

- The meaning of `this` is dynamic - I can only be sure who to call at runtime
- The meaning of `super` is static - I always know who to call in the class after `extends`

Quiz: trace this and super

```
class Counter:
  private var count: Int = 0

  def get: Int = count
  def inc(): Unit = count += 1
  def twice(): Unit =
    this.inc()
    this.inc()
  // ...

class DoubleCounter extends Counter:
  override def inc(): Unit =
    super.inc()
    super.inc()

val c: Counter = new DoubleCounter
c.inc()
c.twice()
```

Trace these calls in order.

1. what is the value of c.get() immediately after c.inc()?

   2

2. what is the value of c.get() after the subsequent c.twice()?

   6

Overloading adds a different signature

- Different parameter types create overloads, static types choose between them.
- The same argument object can select different overloads.
- Return type alone is insufficient.

```
class Counter:
  // ...

  def label(other: Counter): String = "C/C"

class DoubleCounter extends Counter:
  // ...

  def label(other: DoubleCounter): String = "D/D"
```

```
val d = new DoubleCounter
val c: Counter = d

d.label(d)  // "D/D"
d.label(c)  // "C/C"

c eq d      // true
```

Overriding replaces an inherited implementation

- Counter's static API exposes label(Counter); c's runtime class supplies its override.
- DoubleCounter also has a separate overload label(DoubleCounter).

```
class Counter:
  // ...
  def label(other: Counter): String = "C/C"

class DoubleCounter extends Counter:
  // ...
  override def label(other: Counter): String =
    "D/C"

  def label(other: DoubleCounter): String =
    "D/D"
```

```
val d = new DoubleCounter()
val c: Counter = d

c.label(d)                    // "D/C"
d.label(d)                    // "D/D"
```

Single dispatch: signature first, receiver second

- Compile time: select an accessible signature from the receiver's static type
- Use the method name and static argument types to choose an overload
- Runtime: for a virtual call, select the override for that signature
- Use the receiver's runtime class; do not reconsider explicit argument types
- An overloaded method can also be overridden
- Static overload selection does not mean a JVM static method call

Quiz: signature first, receiver second

```
class Counter:
  // ...
  def label(other: Counter): String = "C/C"

class DoubleCounter extends Counter:
  // ...
  override def label(other: Counter): String = "D/C"

  def label(other: DoubleCounter): String = "D/D"

val d: DoubleCounter = new DoubleCounter()
val c: Counter = d
val a: Counter = d
```

All three references point to one object.

`d.label(d)`?

"D/D"

`d.label(a)`?

"D/C"

`c.label(d)`?

"D/C"

`c.label(a)`?

"D/C"

Static vs dynamic dispatch

```
class C:
  def b(): Unit = ...

  private def x(): Unit = ...

  final def y(): Unit = ...

  def z(): Unit =
    this.x()    // static dispatch
    this.y()    // static dispatch
    this.b()    // dynamic dispatch
    C.a()       // static dispatch


object C:
  def a(): Unit = ...
```

- Static the call has a fixed target
- Dynamic the receiver's runtime class selects an override
- On JVM, most dispatch is dynamic with some exceptions
  - private methods
  - final methods
  - methods from companion objects

Example - space simulator

- The simulator contains different space objects that can collide

```
abstract class SpaceObject:
  def collide(other: SpaceObject): String

class Asteroid extends SpaceObject:
  override def collide(o: SpaceObject): String =
    ???

class Spaceship extends SpaceObject:
  override def collide(o: SpaceObject): String =
    ???
```

```
val a: SpaceObject = new Asteroid()
val b: SpaceObject = new Spaceship()

a.collide(b)   // "ship damaged"
b.collide(a)   // "ship damaged"
a.collide(a)   // "rocks fragments"
b.collide(b)   // "ships bounce"
```

First attempt: add a collide overload

This compiles, but does not solve the problem: the argument's runtime class does not select an overload.

```
abstract class SpaceObject:
  def collide(o: SpaceObject): String =
    "generic collision"

class Asteroid extends SpaceObject:
  def collide(o: Spaceship): String =
    "ship damaged"

class Spaceship extends SpaceObject:
  def collide(o: Asteroid): String =
    "ship damaged"
```

```
val a: SpaceObject = new Asteroid()
val b: SpaceObject = new Spaceship()

a.collide(b)    // "generic collision"

val rock = new Asteroid()
val ship = new Spaceship()
rock.collide(ship)  // "ship damaged"
```

Double dispatch: ask the other object

Keep collide(SpaceObject). Its override calls a second method on the other object.

```
abstract class SpaceObject:
  def collide(o: SpaceObject): String
  def hitBy(a: Asteroid): String
  def hitBy(s: Spaceship): String
```

```
class Asteroid extends SpaceObject:
  override def collide(o: SpaceObject): String =
    o.hitBy(this)
  // ... hitBy implementations next

class Spaceship extends SpaceObject:
  override def collide(o: SpaceObject): String =
    o.hitBy(this)
  // ... hitBy implementations next
```

First dispatch: the receiver chooses Asteroid.collide or Spaceship.collide.
Inside each class, this has a specific static type, so the hitBy overload is known.

Double dispatch: all collision bodies

Same SpaceObject API: collide(SpaceObject), hitBy(Asteroid), and hitBy(Spaceship).

```
class Asteroid extends SpaceObject:
  override def collide(o: SpaceObject): String =
    o.hitBy(this)

  override def hitBy(a: Asteroid): String =
    "rocks fragment"

  override def hitBy(s: Spaceship): String =
    "ship damaged"
```

```
class Spaceship extends SpaceObject:
  override def collide(o: SpaceObject): String =
    o.hitBy(this)

  override def hitBy(a: Asteroid): String =
    "ship damaged"

  override def hitBy(s: Spaceship): String =
    "ships bounce"
```

Second dispatch: o's runtime class chooses the override of the selected hitBy signature.
The hitBy parameter records the first kind; the receiver supplies the second kind.

Trace: asteroid collides with spaceship

```
class Asteroid extends SpaceObject:
  override def collide(o: SpaceObject): String =
    o.hitBy(this)
  // ... hitBy methods

class Spaceship extends SpaceObject:
  override def hitBy(a: Asteroid): String =
    "ship damaged"
  // ... collide and hitBy(Spaceship)
```

```
val a: SpaceObject = new Asteroid
val b: SpaceObject = new Spaceship
```

```
a.collide(b)
  → Asteroid.collide(b)
  → b.hitBy(this: Asteroid)
  → Spaceship.hitBy(Asteroid)
  → "ship damaged"
```

1. Dispatch on a → Asteroid.collide; its this is a, and o is b.
2. Dispatch on b → Spaceship.hitBy(Asteroid). Two ordinary virtual calls; no runtime overload selection.

Subtyping polymorphism

Polymorphism

- The many forms in our context means roughly same interface, different underlying data types

- Subtype polymorphism
  - allows a function to take an object of a some type T, but also work correctly, if called with an object of a type S that is a subtype of T

- Parametric polymorphism
  - allows a function or a data type to be written generically, so that it can handle values uniformly without depending on their type

- Ad-hoc polymorphism
  - allows a function to be applied to arguments of different types, but behave differently depending on the type of the argument to which they are applied (e.g., method/operator overloading)

Subtype polymorphism

- A form of type polymorphism
- A relation between two types by some notion of substitutability

If `B` is a subtype of `A`, the subtyping relation

`B <: A` (or in some literature `B` $\sqsubseteq$ `A`)

means that any variable of type `B` can safely be used in any context where type `A` is expected.

Subtyping is relation that is

- transitive
  - if `B <: A` and `C <: B` then `C <: A`
- reflexive
  - `A <: A`

- For every `B <: A` there is an inverse relation `A >: B` means that `A` is a supertype of `B`

Subtype polymorphism

If B is a subtype of A means that
- any variable of type B can safely be used in any context where type A is expected
- everything one can do with a value of A one should also be able to do with a value of B

What are the contexts?

```
class Counter: // ...
class DoubleCounter extends Counter:  // ...

def tick(c: Counter): Int =
  c.inc()

val d = new DoubleCounter
val c: Counter = d              // assignment
tick(d)                         // argument

def fresh(): Counter =
  new DoubleCounter()           // result
```

1. Variable Assignment

A variable of a given type may be assigned a value of any subtype of that type.

2. Method Invocation & Return

Invoked with a subtype argument, or returning a subtype instance.

A subtype can be used through a supertype view

```
class Counter:
  // ...

class DoubleCounter extends Counter:
  // ...

def tick(c: Counter): Int =
  c.inc()
  c.get

val d = new DoubleCounter
val c: Counter = d            // assignment
tick(d)                       // argument

def fresh(): Counter =
  new DoubleCounter()         // result
```

- `DoubleCounter <: Counter` is the declared subtype relation.
- Assignments, arguments, and results accept a subtype value.
- The static interface guarantees which operations exist.
- It does not by itself guarantee the behavior of overrides.

Subtype polymorphism

- How do we create subtypes?
- In class-based object-oriented programming languages primarily by
  - subclassing
  - interface / traits implementation

```
class A
class B extends A
```

```
interface A { … }
class B implements A { … }
```



- It does not always create a subtype (non-public extends in C++ does not create a subtype)

```
class A {};
class B : public A {};

// does not create a subtype
class B : A {};
```



Subtype polymorphism - other ways

- Structural subtyping: Go, TypeScript, Scala 3
- Variance:
  - Java (? extends T),
  - Scala (+T / -T),
  - C# (out T / in T)
- Coercion (implicit conversion):
  - Scala (implicits, given),
  - C++ (user-defined conversions),
  - Rust (From/Into traits)
- Refinement types: Scala (refinement types), TypeScript (intersection & narrowed types)
- Union types: Scala 3, TypeScript, OCaml
- Record Width and Depth Subtyping: TypeScript, OCaml
- ...

Subclassing, inheritance, and subtyping

- Subclassing is a declaration: DoubleCounter extends Counter.
- Inheritance reuses accessible members and implementation.
- Inherited private state still exists, but subclasses cannot access it directly.
- Overriding replaces behavior; the base implementation can still be reached via super.
- Nominal subtyping is the type relation introduced by the declaration.
- Behavioral subtyping asks whether all promised client-visible behavior is preserved.
- A compiler can establish the nominal relation without proving the behavioral one.

Liskov Substitution Principle (LSP)



B. Liskov

"Let Φ(x) be a property provable about objects x of type A. Then Φ(y) should be true for objects y of type B where B is a subtype of A (B <: A)."

-- in Behavioral Notion of Subtyping, Barbara Liskov and Jeannette Wing, ACM TOPLAS 16 (6), 1994

- A behavioral subtype preserves the promises clients rely on.
- A value of B can stand in for A without breaking properties guaranteed by A's contract.
- The compiler's nominal subtype check does not prove this behavioral relationship.

Subclassing

- A subclass inherits all protected and public members
  - no private (because you do not even know that they are in)
  - no overridden members (because you change the definition)
- It has to call superclass constructor (implicitly or explicitly)
  - this in turn invokes the whole constructor chain

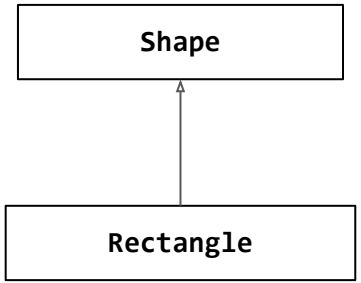
Benefits of subclassing

- Simplification
  - in implementation
    - code reuse, fix bugs once
    - no client code changes to use subclasses
  - in specification
    - once you understand the superclass you only need to familiarize with changes
    - differences not buried under mass of similarities

Inheritance example

```
abstract class Shape

class Rectangle(width: Int, height: Int) extends Shape:
  def resize(w: Int, h: Int): Unit = { … }
```

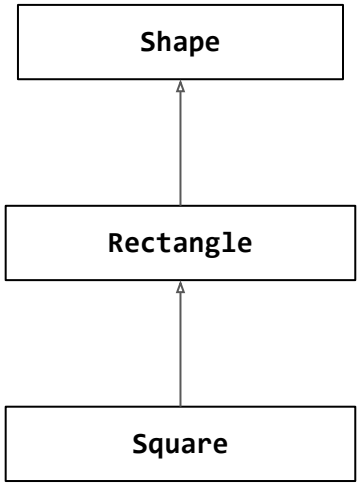


Inheritance example

```
abstract class Shape

class Rectangle(width: Int, height: Int) extends Shape:
  def resize(w: Int, h: Int): Unit = { … }


class Square(side: Int) extends Rectangle(side, side):
  override def resize(w: Int, h: Int): Unit = {
    ???
  }
```



How to add a square? What to do with the resize method?

1. effects: fits shape to given size
   errors if dw != dh

2. effects: sets this.width and this.height to dw

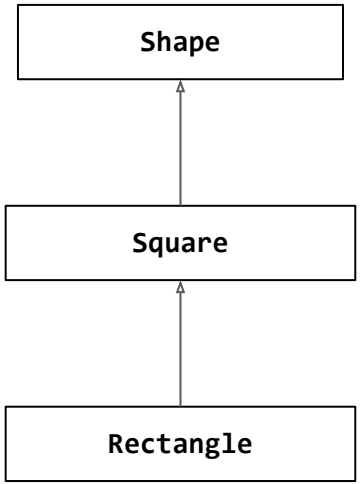
Inheritance example

Is it the other way around, a rectangle is a generalization of a square?

```
abstract class Shape

class Square(s: Int) extends Shape:
  def resize(d: Int): Unit = { … }


class Rectangle(w: Int, h: Int) extends Square(???):
```



But then what to do in the constructor?

Behavioral subtyping

- Perhaps it does not work because the example in contrived.
- What if we do not have `resize` and instead individual `setWidth` / `setHeight` methods?
- Now we should be able to fit a `Square` rather easily, right?

```
class Rectangle(var width: Int, var height: Int)
  extends Shape:

  def setWidth(x: Int): Unit = this.width = x

  def setHeight(x: Int): Unit = this.height = x

  def area: Double = this.height * this.width
```

```
class Square(size: Int) extends Rectangle(size, size):

  override def setWidth(x: Int) =
    this.width = x
    this.height = x


  override def setHeight(x: Int) =
    this.width = x
    this.height = x
```

Behavioral subtyping

```
def test(r: Rectangle) {
  r.setWidth(5)
  r.setHeight(4)

  assert(r.area() == 20)
}
```

```
test(Rectangle(1, 2))
```



```
test(Square(1))
```

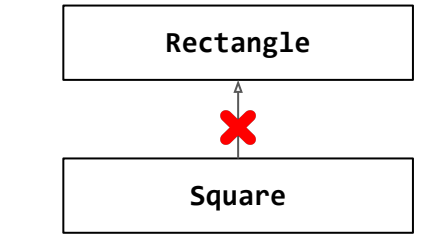


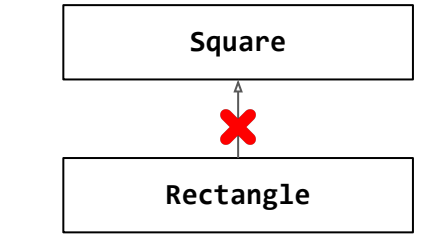
Violating behavioral subtyping. Subclasses should satisfy the expectations of clients accessing subclass objects through references of superclass type. This goes beyond just syntactic safety (e.g. method-not-found errors) but also as regards behavioral correctness.

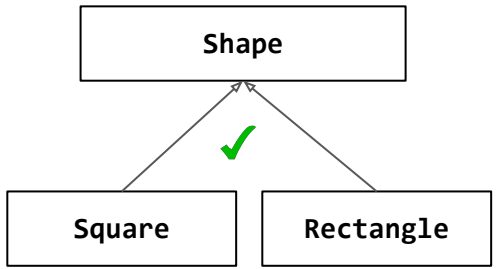
Unrelated subtypes

- `Square` is not a true subtype of `Rectangle`
  - rectangles width and height are expected to be changed independently
  - square violates this expectations
- `Rectangle` is not a true subtype of `Square`
  - square is expected to have just one dimension
  - rectangle violates this expectations
- They are unrelated types!

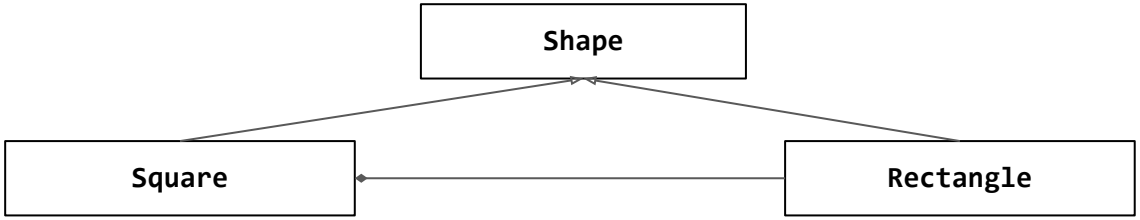
- Key takeaway: subtyping is not always intuitive
  - easy to make a mistake
  - yet it forces us to think!







Inheritance example - solution



- Flat hierarchy, possibly using composition to reuse behavior already implemented in the Rectangle by composition

```
abstract class Shape

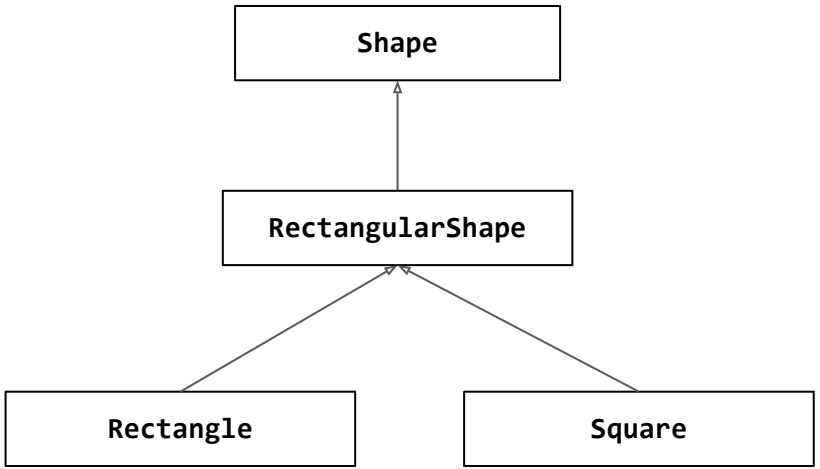
class Square(s: Int) extends Shape:

  private var delegate = new Rectangle(s, s)

  override def draw(canvas: Canvas): Square = {
    // drawing a square is the same as drawing a rectangle
    // so why not reuse it
    delegate.draw(canvas)
  }
```

Inheritance example - alternative solution

- If and only if there is something to be shared given that they are both rectangular shapes, we could extract a common superclass.



The contract determines substitutability

```
class Counter:

  private var count: Int = 0

  def get: Int = count

  def set(n: Int): Unit =
    require(n >= 0)
    count = n

  def inc(): Unit = count += 1
  // ...

def tick(c: Counter): Unit =
  val before = c.get
  c.inc()
  assert(c.get >= before + c.step)
```

- Contract for `get`: return the current nonnegative count.
- Contract for `set(n)`: accept every n ≥ 0; afterward get == n.
- Contract for `inc()`: increase get by at least step; assume no overflow.
- DoubleCounter's two steps satisfy this contract.
- An exactly-one-step promise would be violated.
- Matching signatures alone does not prove substitutability.

A bounded counter exposes a contract conflict

- A subtype must honor the Counter contract even when seen only as a Counter.
- Making the increment conditional violates the promised progress at the limit.

```
class Counter:
  private var count: Int = 0
  def get: Int = count
  def inc(): Unit = count += 1
  // ...

class BoundedCounter(limit: Int)
  extends Counter(0):

  require(limit >= 0)

  override def inc(): Unit =
    if get < limit then super.inc()
```

```
val c: Counter = new BoundedCounter(0)
val before = c.get
c.inc()

c.get                          // 0
c.get >= before + c.step       // false
```

Quiz: can inheritance preserve this bound?

```
class Counter(init: Int = 0,
              val step: Int = 1):
  private var count: Int = init
  def get: Int = count
  def set(n: Int): Unit =
    require(n >= 0)
    count = n
  def inc(): Unit = count += step
  // ... constructor validation

class BoundedCounter(limit: Int)
  extends Counter(0):
  require(limit >= 0)
  override def inc(): Unit =
    if get < limit then super.inc()

val c: Counter = new BoundedCounter(10)
c.set(20)
```

Counter.set(n) accepts every n ≥ 0 and promises get == n afterward.

1. May BoundedCounter reject set(20)?

No: that strengthens the precondition.

2. May it silently clamp the result to 10?

No: that violates the promised result.

3. Can it honor Counter and guarantee get ≤ 10?

No: the two contracts conflict for set(20).

Use composition when the public contracts differ

```
class Counter(init: Int = 0,
              val step: Int = 1):
  private var count: Int = init
  def get: Int = count
  def inc(): Unit = count += step
  // ... validation and set

class BoundedCounter(val limit: Int):
  require(limit >= 0)
  private val inner = new Counter()
  def get: Int = inner.get
  def tryInc(): Boolean =
    if get < limit then
      inner.inc()
      true
    else
      false
```

- The bounded counter contains a Counter; it is not its subtype.
- It exposes a different operation: `tryInc()` may report failure.
- It does not expose the unrestricted `set` method.
- Implementation is reused without inheriting the conflicting API.
- Composition makes the boundary explicit.

Behavioral subtyping: obligations on overrides

- Accept at least every input accepted by the base contract: no stronger precondition.
- Guarantee at least the base result: no weaker postcondition.
- Preserve invariants and any promised limits on side effects.
- New methods must also respect invariants and documented history constraints.
- Unexpected failures can violate a behavioral contract even if the compiler permits them.
- Java additionally restricts checked exceptions in overriding declarations.
- Scala does not have Java-style checked-exception enforcement.

Substitution principle for methods

- Constraints on methods
  - For each supertype method, subtype must have such a method
  - Could be inherited or overridden
- Each overriding method must strengthen (or match) the spec:
  - Weaker precondition = Ask nothing extra of client
    - Requires clause is at most as strict as in supertype's method
  - Stronger postcondition = Guarantee at least as much
    - Effects clause is at least as strict as in the supertype method
    - Promise more (or the same) in returns clause
    - Throws clause must indicate fewer (or same) possible exception types

https://scastie.scala-lang.org/N2BJcBpfTbqi4R1vmHuupQ

Example

- Given the substitution principle, which types can I use for the return type in an overridden method?

```
class A:
  def f() = 1

class B extends A:
  def g() = 1

class C extends B:
  def h() = 1

class T1:
 def m(x: B): B = new B
```

```
class T2 extends T1:
  override def m(x: B): ? = ...
```

https://scastie.scala-lang.org/N2BJcBpfTbqi4R1vmHuupQ

Example

- Given the substitution principle, which types can I use for the return type in an overridden method?

```
class A:
  def f() = 1

class B extends A:
  def g() = 1

class C extends B:
  def h() = 1

class T1:
 def m(x: B): B = new B
```

```
class T2 extends T1:
  override def m(x: B): B = ...  // OK
```

```
def f(t: T1) = {
  t.m(new B).g()
}


f(new T2)
```

https://scastie.scala-lang.org/N2BJcBpfTbqi4R1vmHuupQ

Example

- Given the substitution principle, which types can I use for the return type in an overridden method?

```
class A:
  def f() = 1

class B extends A:
  def g() = 1

class C extends B:
  def h() = 1

class T1:
 def m(x: B): B = new B
```

```
class T2 extends T1:
  override def m(x: B): C = ...  // OK
```

```
def f(t: T1) = {
  t.m(new B).g()
}                  // A subtype has all methods of the
                   // super types, C does have g


f(new T2)
```

https://scastie.scala-lang.org/N2BJcBpfTbqi4R1vmHuupQ

Example

- Given the substitution principle, which types can I use for the return type in an overridden method?

```
class A:
  def f() = 1

class B extends A:
  def g() = 1

class C extends B:
  def h() = 1

class T1:
 def m(x: B): B = new B
```

```
class T2 extends T1:
  override def m(x: B): A = ...  // BAD
```

```
def f(t: T1) = {
  t.m(new B).g()    // A does not have method g
}


f(new T2)
```

https://scastie.scala-lang.org/N2BJcBpfTbqi4R1vmHuupQ

Example

- Given the substitution principle, which types can I use for the parameter in an overridden method?

```
class A:
  def f() = 1

class B extends A:
  def g() = 1

class C extends B:
  def h() = 1

class T1:
 def m(x: B): B = new B
```

```
class T2 extends T1:
  override def m(x: ?): B = ...
```

https://scastie.scala-lang.org/N2BJcBpfTbqi4R1vmHuupQ

Example

- Given the substitution principle, which types can I use for the parameter in an overridden method?

```
class A:
  def f() = 1

class B extends A:
  def g() = 1

class C extends B:
  def h() = 1

class T1:
 def m(x: B): B = new B
```

```
class T2 extends T1:
  override def m(x: B): B = {   // OK
    x.g()
  }
```

```
def f(t: T1) = {
  t.m(new B)
}


f(new T2)
```

https://scastie.scala-lang.org/N2BJcBpfTbqi4R1vmHuupQ

Example

- Given the substitution principle, which types can I use for the parameter in an overridden method?

```
class A:
  def f() = 1

class B extends A:
  def g() = 1

class C extends B:
  def h() = 1

class T1:
 def m(x: B): B = new B
```

```
class T2 extends T1:
  override def m(x: C): B = {   // BAD
    x.h()
  }
```

```
def f(t: T1) = {
  t.m(new B)
}


f(new T2)
```

https://scastie.scala-lang.org/N2BJcBpfTbqi4R1vmHuupQ

Example

- Given the substitution principle, which types can I use for the parameter in an overridden method?

```
class A:
  def f() = 1

class B extends A:
  def g() = 1

class C extends B:
  def h() = 1

class T1:
 def m(x: B): B = new B
```

```
class T2 extends T1:
  override def m(x: A): B = {   // OK
    x.f()
  }
```

```
def f(t: T1) = {
  t.m(new B)
}


f(new T2)
```

https://scastie.scala-lang.org/N2BJcBpfTbqi4R1vmHuupQ

Example

- Given the substitution principle, which types can I use for the parameter in an overridden method?

```
class A:
  def f() = 1

class B extends A:
  def g() = 1

class C extends B:
  def h() = 1

class T1:
 def m(x: B): B = new B
```

```
class T2 extends T1:
  override def m(x: A): B = {   // BAD
    x.f()
  }
```

```
def f(t: T1) = {
  t.m(new B)
}


f(new T2)
```

- Conceptually it would make sense to be able to use super type, but
- Java/Scala/C++ do not allow contravariance in override method parameters because of overloading

Substitution principle for methods

```
class A
class B extends A
class C extends B

class T1 {
 def m(x: B): B = new B
}

class T2 extends T1 {
 // OK - simply changing the code
 override def m(x: B): B = ...

 // OK - we can provide more
 // (anything that can be done with B can be done with C)
 override def m(x: B): C = ...

 // ERROR - A is not a subtype of B
 override def m(x: B): A = ...

 // ERROR - it is not an override but overload
 // (but without overload it would make sense:
 //  anything that can be done with A can be done with B)
 override def m(x: A): B = ...
}
```

- Parameter types are in contravariant position
  - could be replaced by supertypes (as this does not place any extra demand on clients)
  - does not work in Java/Scala/C++ because of overloading
- Return type is in covariant position
  - can be replaced by subtype
  - because anything can could be done with the supertype could be done with subtype
- Exceptions
  - no new exceptions
  - existing can be replaced by subtypes
  - existing can be removed as this will not change what client expects

Subtyping schemes

Type systems classification regarding subtyping

- Nominal typing
  - type equivalence and compatibility is determined by explicit declarations
- Structural typing
  - type equivalence and compatibility is determined by type's actual structure

- In this course we focus primarily on static type systems with nominal typing
  - Many of the mainstream OOP languages fall in this category (C++, Java, Kotlin, Scala)

- Duck typing
  - type equivalence and compatibility is determined by the duck test:

    "If it walks like a duck and it quacks like a duck, then it must be a duck"
  - looks like structural typing, but it is a runtime characteristics (e.g., JavaScript, Python)

Nominal typing

```
abstract class Animal:
  def speak: String

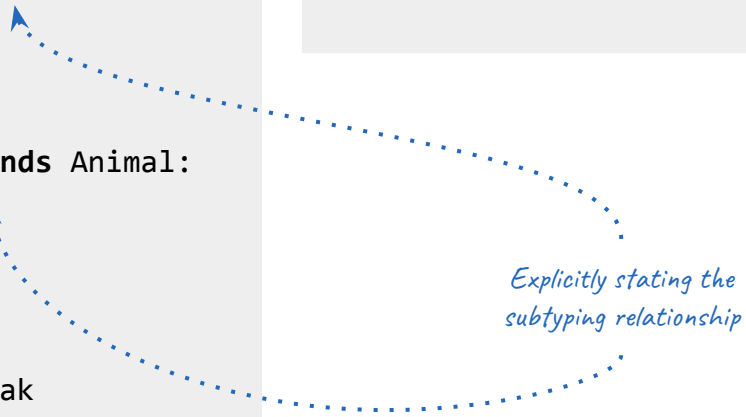

class Cat(name: String) extends Animal:
  override def speak =
    this.name + " mnau!"


class Dog(name: String) extends Animal:
  override def speak =
    this.name + " haf!"


def speak(pet: Animal) =
    "Animal says " + pet.speak
```

```
val cat = new Cat("Garfield")
speak(cat)     // Animal says Garfield mnau!

val dog = new Dog("Odie")
speak(dog)     // Animal says Odie haf!
```



Structural typing



```
type Animal interface {
    Speak() string
}

type Dog struct {
    name string
}

type Cat struct {
    name string
}

func (self Dog) Speak() string {
    return self.name + " hafhaf!"
}

func (self Cat) Speak() string {
    return self.name + " mnau!"
}

func Speak(a Animal) string {
    return "Animal says " + a.Speak()
}
```

No relationship with Animal



```
Cat cat = Cat { "Garfield" }
fmt.Println(Speak(cat))    // Animal says Garfield mnau!

Dog dog = Dog { "Odie" }
fmt.Println(Speak(dog))    // Animal says Odie haf!
```

Structural typing

```ts
interface Animal {
  speak(): string;
}

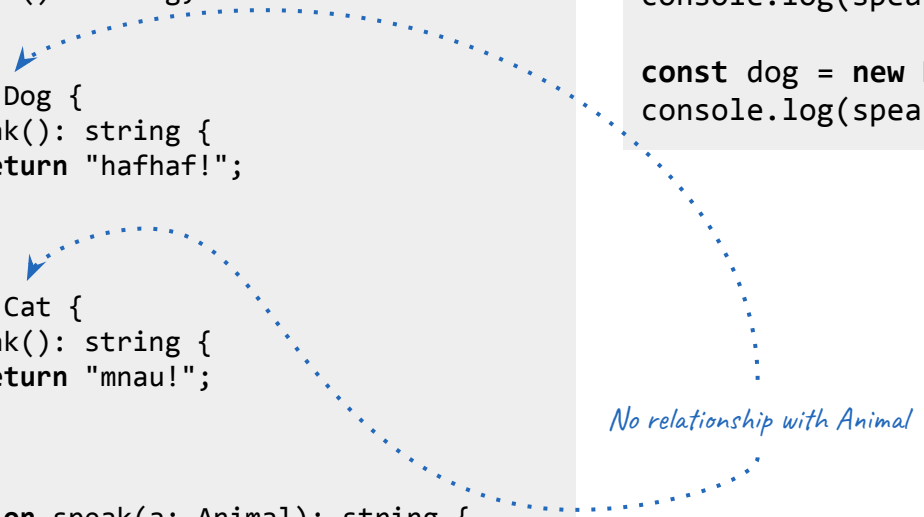
class Dog {
  speak(): string {
    return "hafhaf!";
  }
}

class Cat {
  speak(): string {
    return "mnau!";
  }
}

function speak(a: Animal): string {
  return "Animal says " + a.speak();
}
```

```ts
const cat = new Cat();
console.log(speak(cat)); // Animal says mnau!

const dog = new Dog();
console.log(speak(dog)); // Animal says hafhaf!
```



https://scastie.scala-lang.org/NMOCmN7MThaPaqrgyyxTRw

Structural typing in Scala

```
import
scala.reflect.Selectable.reflectiveSelectable


type Animal = {
  def speak: String
}

class Cat(name: String):
  def speak =
    this.name + " mnau!"

class Dog(name: String):
  def speak =
    this.name + " haf!"

def speak(pet: Animal) =
    "Animal says " + pet.speak
```

No relationship to one another

```
val cat = new Cat("Garfield")
speak(cat)     // Animal says Garfield mnau!

val dog = new Dog("Odie")
speak(dog)     // Animal says Odie haf!
```

- Scala supports structural typing uses reflection

https://scastie.scala-lang.org/BIvcWabATXCG33yFncQf5g

Structural typing in Scala

```
import
scala.reflect.Selectable.reflectiveSelectable


class Cat(name: String):
  def speak =
    this.name + " mnau!"


class Dog(name: String):
  def speak =
    this.name + " haf!"

def speak(pet: { def speak: String }) =
    "Animal says " + pet.speak
```

```
val cat = new Cat("Garfield")
speak(cat)     // Animal says Garfield mnau!

val dog = new Dog("Odie")
speak(dog)     // Animal says Odie haf!
```

- Scala supports structural typing uses reflection

Duck typing

```
class Cat:
    def __init__(self, name):
        self.name = name

    def speak(self):
        return self.name + " mnau!"

class Dog:
    def __init__(self, name):
        self.name = name

    def speak(self):
        return self.name + " haf!"


def speak(pet):
    "Animal says " + pet.speak()
```



```
cat = Cat("Garfield")
speak(cat)     # Animal says Garfield mnau!

dog = Dog("Odie")
speak(dog)     # Animal says Odie haf!
```

If it walks like a duck, quacks like a duck, then it must be a duck

Compile-time duck typing (CTDT)



```
class Dog {
public:
  Dog(std::string const& name): name{name} {}

    std::string speak() const {
        return name + " says haf!";
    }

private:
    std::string name;
};


class Cat {
public:
    Cat(std::string const& name): name{name} {}

    std::string speak() const {
        return name + " says mnau!";
    }

private:
    std::string name;
};
```

```
template<typename T>
std::string speak(const T& pet) {
    return pet.speak();
}

int main() {
    Cat cat("Garfield");
    std::cout << speak(cat) << std::endl;

    Dog dog("Odie");
    std::cout << speak(dog) << std::endl;

    return 0;
}
```

C++ concepts

```cpp
// 1. Define explicit concept
template<typename T>
concept Speakable = requires(T pet) {
  { pet.speak() } -> std::convertible_to<std::string>;
};

// 2. Constrain template using concept
template<Speakable T>
std::string speak(const T& pet) {
  return pet.speak();
}

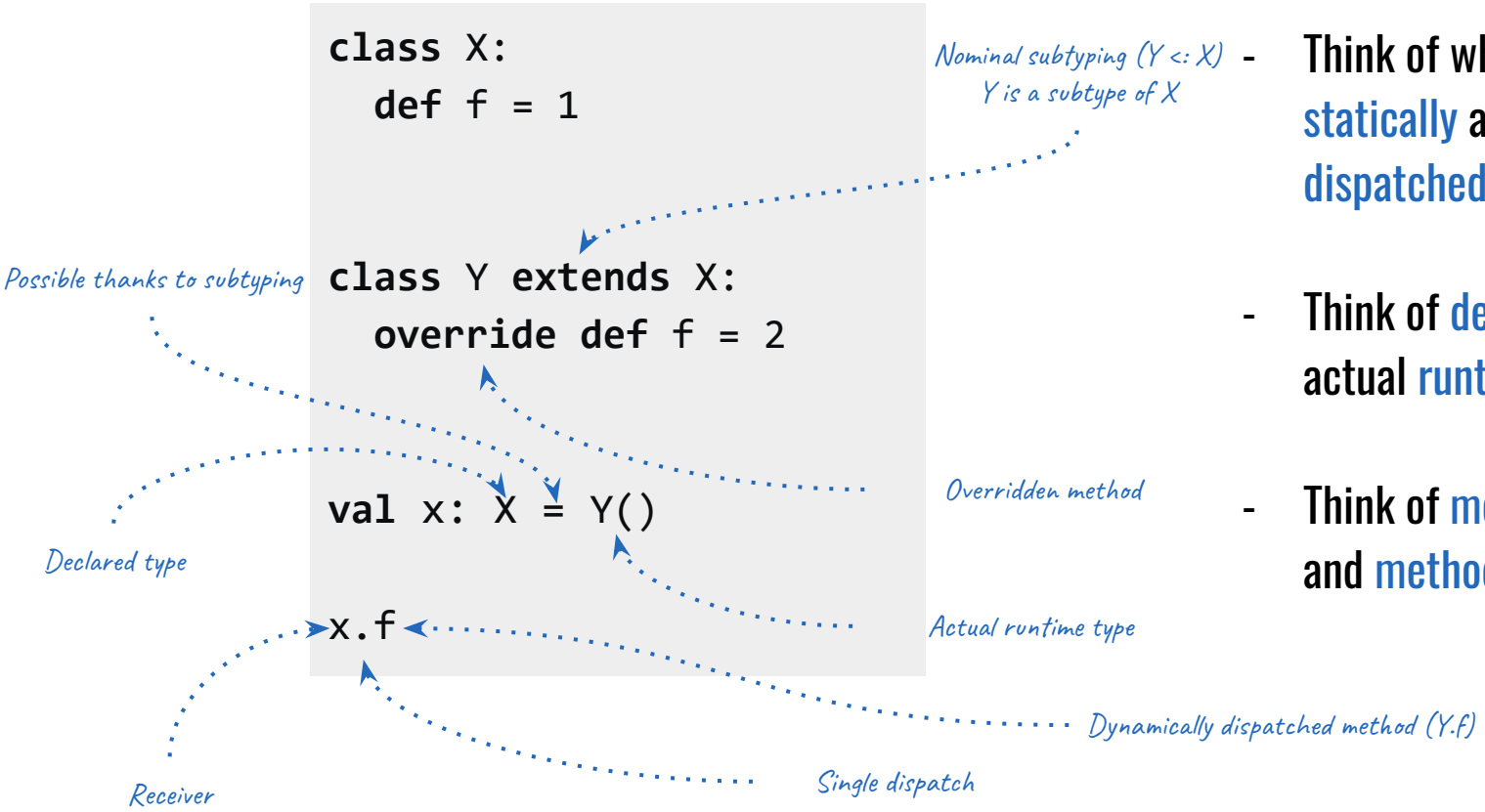
// Usage
int main() {
  Cat cat("Garfield");
  speak(cat); // OK: satisfies Speakable

  int num = 42;
  // speak(num);
  // Compile error: 'int' does not
  // satisfy concept 'Speakable'
}
```

- Implicit vs. Explicit Intent
  - CTDT: Requirements are implicit; checks happen only when methods are called inside template bodies.
  - Concepts: Requirements are explicitly declared beforehand as a formal interface contract.
- Compiler Error Quality
  - CTDT: Deep, cryptic template instantiation error call-stacks.
  - Concepts: Clear, short error messages right at the call site stating which requirement failed.
- Overloading & Specialization
  - Functions can be cleanly overloaded based on concept constraints.
- Compilation Speed
  - Concepts fail fast during constraint checking before trying full template instantiation.

Summary

Summary



- Think of what is dispatched statically and what is dispatched dynamically
- Think of declared type vs actual runtime type
- Think of method overloading and method overriding

Subtyping

- A type relation between two types, usually written B <: A.
- It means: every value of type B can safely be used wherever a value of type A is expected.
- Different ways of establishing subtyping
- In mainstream OOP, it is often done by subclassing

Subclassing

A syntactic / code-reuse mechanism where one class is declared to be a subclass of another.
It usually means:

- The subclass inherits the members (fields, methods) of the superclass.
- It can add or override members.

Relation to subtyping:

- In many OOP languages (Java, C#), subclassing automatically induces subtyping: if class B extends A, then B <: A.
- But in principle, subclassing ≠ subtyping. (A language could allow code reuse without establishing a subtype relation.)

Inheritance

- A mechanism for code reuse where a class (or trait) reuses the definitions (implementation, fields, methods) of another.
- Two kinds (more about it later)
  - Implementation inheritance: reuse concrete code
  - Interface inheritance: reuse only the specification
- Subclassing is the declaration that sets up inheritance.
- Inheritance is the actual process of reusing the code or interface.

Questions to ask at every call

- What is the receiver's static type, if the language has one?
- What are the static types of the explicit arguments?
- Which method signature does the language select?
- Is the call virtual, and what is the receiver's runtime class?
- Does the selected implementation preserve the client's contract?

- Keep object identity, member lookup, dispatch, and substitutability separate.

Bonus

Example - field resolution in Java



```
class A {
  int x = 1;
}

class B extends A {
  int x = 2;
}


B b = new B();
A ab = b;

b.x;          // 2
ab.x;         // 1
((B)ab).x;    // 2
```

- In Java, fields are resolved statically

Example - field resolution in Java



```
class A {
  int x = 1;
}

class B extends A {
  int x = 2;
}


B b = new B();
A ab = b;

b.x;          // 2
ab.x;         // 1
((B)ab).x;    // 2
```

```
class A {
  int x = 1;

  int getX() {
    return x;
  }
}

class B extends A {
  int x = 2;
}


B b = new B();
A ab = b;

b.getX();          // 1
ab.getX();         // 1
((B)ab).getX();    // 1
```

- In Java, fields are resolved statically

Field resolution in Scala

```
class A {
  val x = 1
}

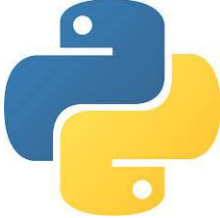
class B extends A {
  override val x = 2
}

val b: B = B()
val ab: A = b

b.x               // 2
ab.x              // 2
```

- Scala use uniform access principle
  - Fields and methods share the same namespace
  - Fields are resolved dynamically

Variable binding in Python



```
def create_multipliers():
    multipliers = []

    for i in range(5):
        def multiplier(x):
            return i * x

        multipliers.append(multiplier)

    return multipliers

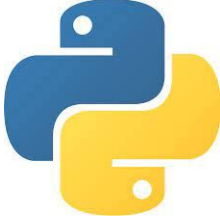

for multiplier in create_multipliers():
    print(multiplier(2))
```

```
0    8
2    8
4    8
6    8
8    8
```

Python's closures are late binding.
Values of variables used in closures are looked up at the time the closure is called.

https://docs.python-guide.org/writing/gotchas/

Variable binding in Python



```
def create_multipliers():
    multipliers = []

    for i in range(5):
        def multiplier(x, i=i):
            return i * x

        multipliers.append(multiplier)

    return multipliers


for multiplier in create_multipliers():
    print(multiplier(2))
```

Python's default arguments are evaluated once at the function defined.

```
0    8
2    8
4    8
6    8
8    8
```

Python's closures use late binding.
Values of variables used in closures are looked up at the time the closure is called.

https://docs.python-guide.org/writing/gotchas/